Second Implementation Iteration: One-Time Classroom Allocation

with Availability and Capacity Rules

Software Design / Iteration Report

Name: Najib Attar
Date: March 2026

1. Iteration Goal

The goal of this second implementation iteration is to extend the first prototype by adding a second decision rule to the allocation process. In Iteration 1, the system evaluated a one-time classroom request using only the `AvailabilityRule`. In this iteration, the system also applies a `CapacityRule` to verify whether the requested classroom can accommodate the number of participants.

This iteration preserves the same request type and resource type while making the decision mechanism more realistic. A request is approved only if the classroom is both available during the requested time slot and has sufficient capacity.

2. Changes from Iteration 1

Compared with the first iteration, the following changes were introduced:

- Added a new rule: `CapacityRule`
- Added a new request attribute: `participantCount`
- Updated the rule evaluation process so that approval now depends on two rules instead of one

The system now evaluates a request according to the following condition:

$$RequestApproved = AvailabilityRule \wedge CapacityRule$$

3. Scenario

A user submits a one-time classroom request for a given time slot and specifies the expected number of participants. The system checks whether the requested classroom is

available during that time slot and whether the classroom capacity is sufficient for the participant count.

If both conditions are satisfied, an allocation is created. Otherwise, the request is rejected.

4. Modules and Their Tasks

A. Request Management Module

- Receive the one-time classroom request from the user
- Create and store `OneTimeRequest`
- Include request details:
  - requester
  - requested classroom
  - requested time slot
  - participant count
- Send the request to the Rule Evaluation Module

B. Resource Management Module

- Manage classroom information
- Provide classroom details when needed
- Provide classroom capacity information

C. Rule Evaluation Module

- Evaluate the request using `AvailabilityRule`
- Evaluate the request using `CapacityRule`
- Check whether the classroom is already allocated during the requested time slot
- Check whether the classroom capacity is sufficient for the participant count
- Return the final decision: Approved or Rejected

D. Allocation Management Module

- Create `Allocation` if all rules pass
- Store the new allocation
- Update request status to Approved
- Reject the request if one or more rules fail
- Update request status to Rejected

5. Iteration Comparison

Iteration	Request Type	Resource Type	Rules Implemented
Iteration 1	OneTimeRequest	Classroom	AvailabilityRule
Iteration 2	OneTimeRequest	Classroom	AvailabilityRule + CapacityRule

6. Updated Class Diagram

Figure 1 shows the updated simplified class diagram for the second implementation iteration.

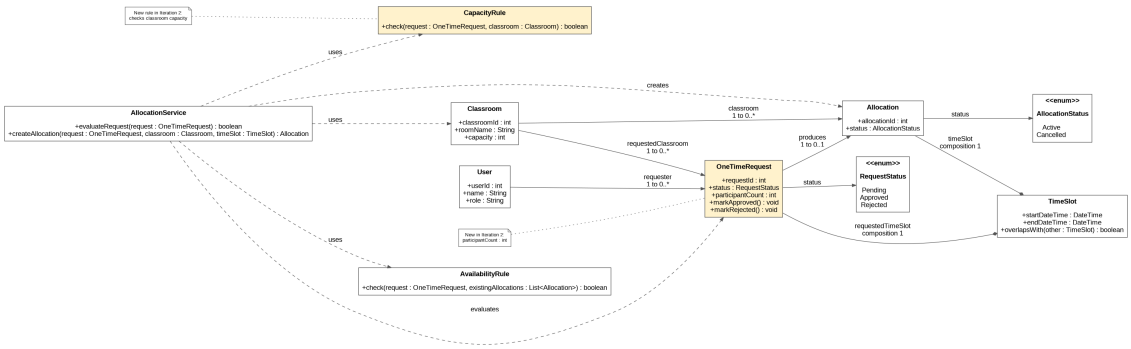

Figure 1: Updated Simplified Class Diagram

7. Updated Workflow Diagram

Figure 2 shows the workflow for the second iteration of the one-time classroom allocation process.

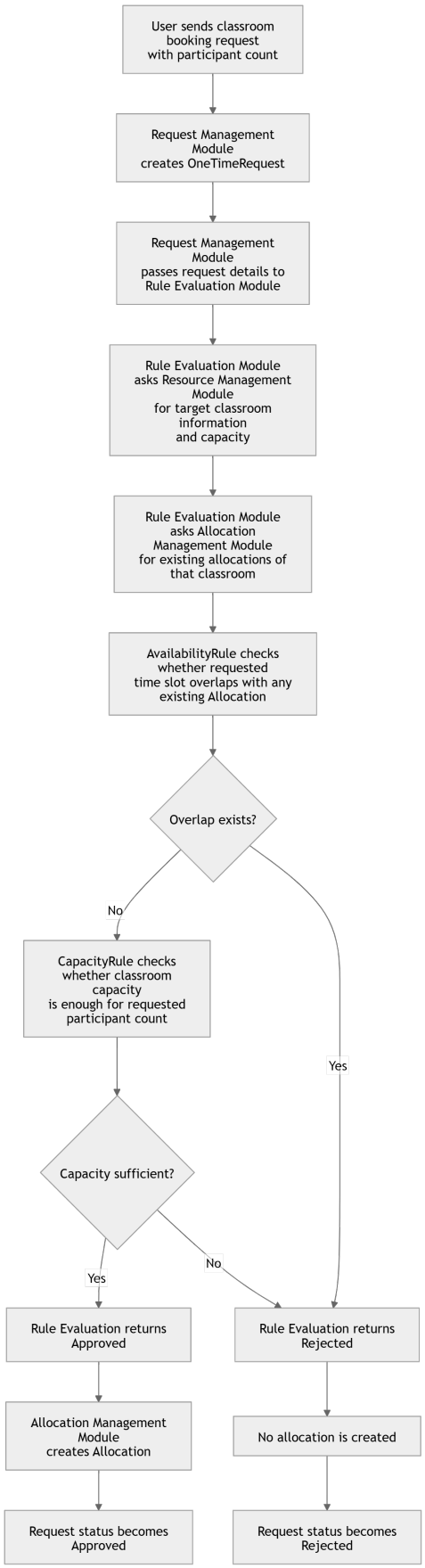

Figure 2: Workflow Diagram

8. Prototype Implementation

To validate the design of this iteration, the prototype implemented in C++ was extended by adding a new `CapacityRule` and a new request attribute, `participantCount`. The allocation service now evaluates each request using two rules:

- `AvailabilityRule`: checks whether the requested classroom is already allocated during an overlapping time slot
- `CapacityRule`: checks whether the classroom capacity is sufficient for the required participant count

The request is approved only if both rules pass. This implementation demonstrates that the system can now reject requests either due to time-slot conflict or due to insufficient classroom capacity.

9. Summary

This second implementation iteration extends the first prototype by adding a second rule to the decision mechanism while preserving the same request type and resource type. The system now evaluates one-time classroom requests based on both time-slot availability and classroom capacity.

This iteration demonstrates the iterative development approach by expanding the system in a controlled way without changing the overall architecture. Future iterations can continue this process by introducing additional request types, resource types, and decision rules.